ТУСУР
TUSUR UNIVERSITY

Архитектуры нейронных сетей для сегментации и детектирования объектов

ТУСУР
TUSUR UNIVERSITY

Типовые архитектуры нейронных сетей, используемых для сегментации изображений

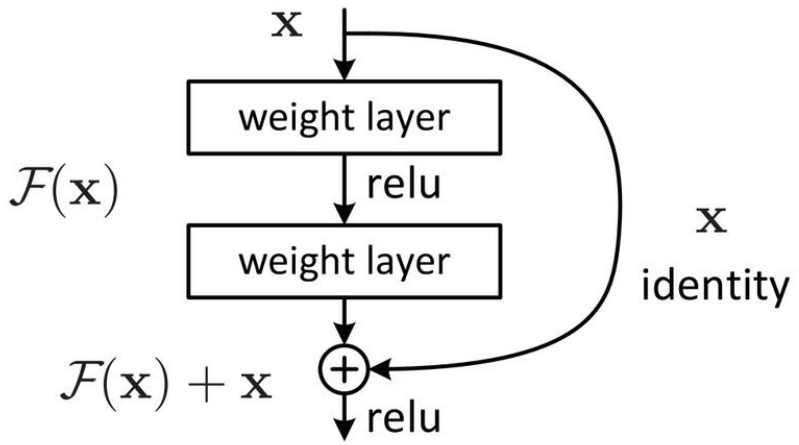
Сверточная нейронная сеть

Сверточная нейронная сеть (*convolutional neural network*, *CNN*) — специальная архитектура нейронных сетей, предложенная Яном Лекуном, изначально нацеленная на эффективное распознавание изображений.

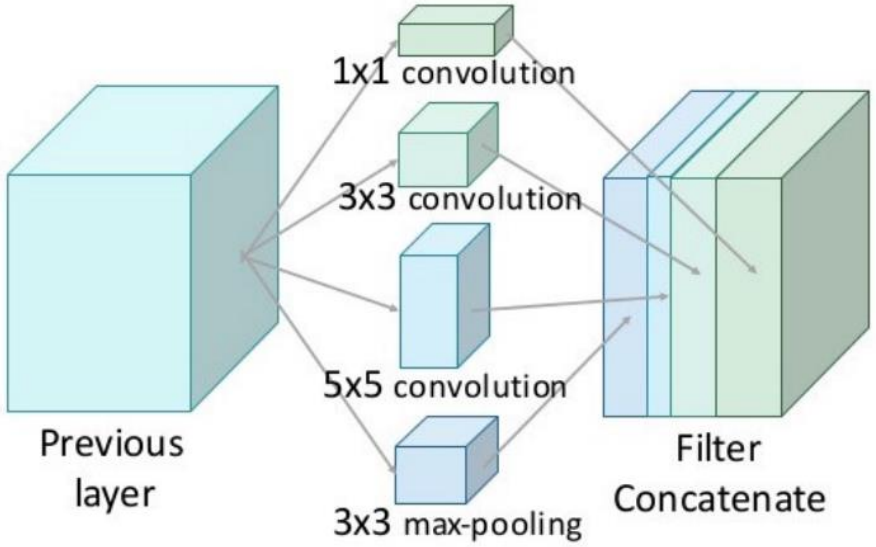
Основные виды слоёв в CNN:

- Свёрточный слой;
- Пулинговый слой;
- Начальный (inception) модуль
- Остаточный (residual) модуль

Residual модуль

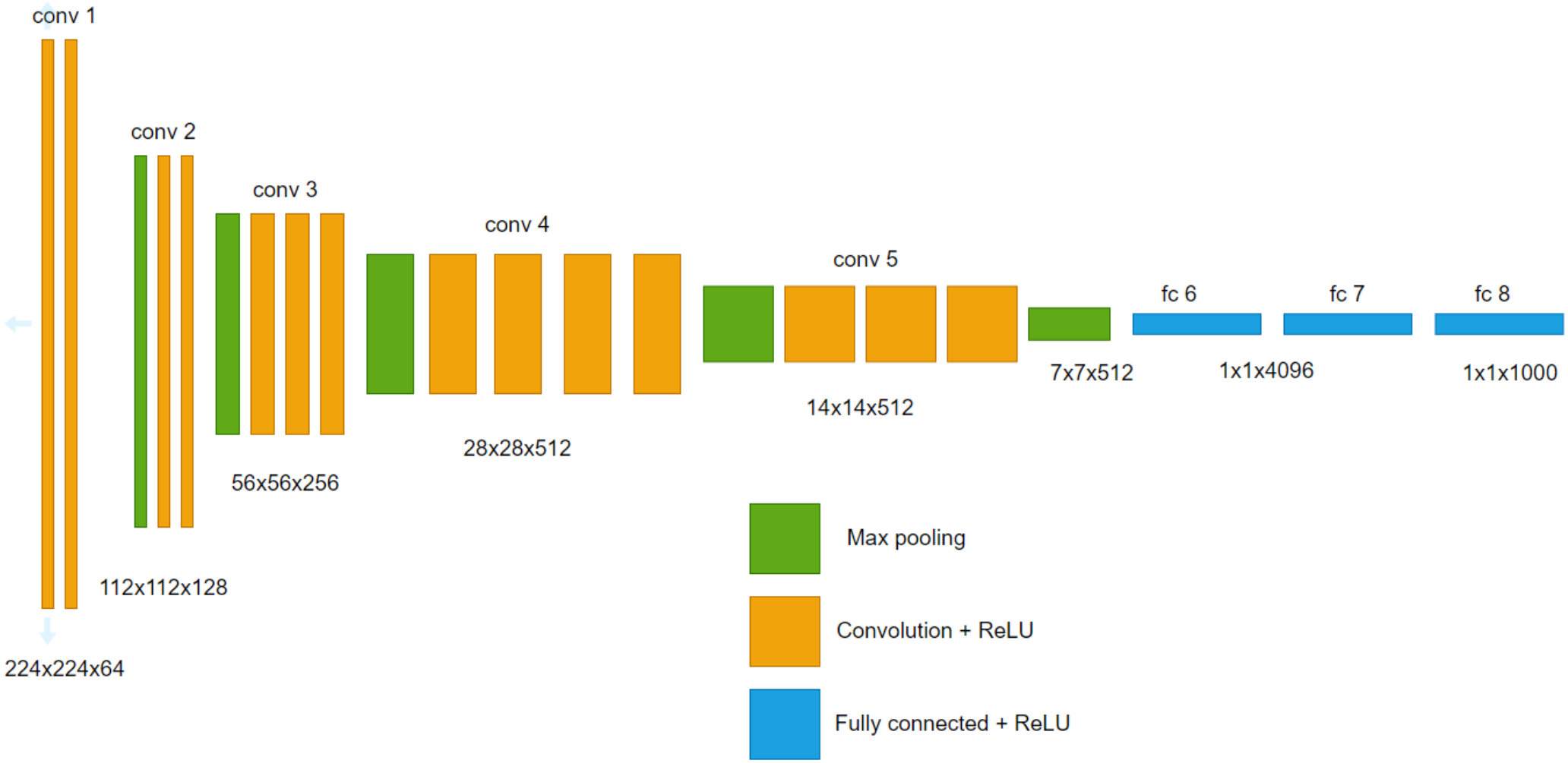


Inception модуль



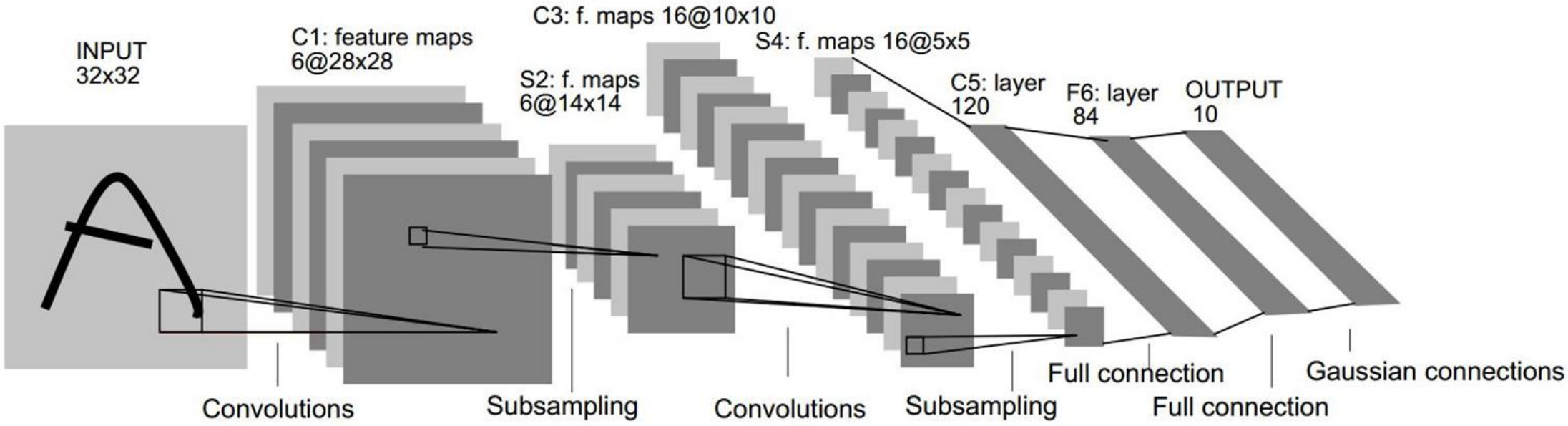
VGG19

TUSUR UNIVERSITY



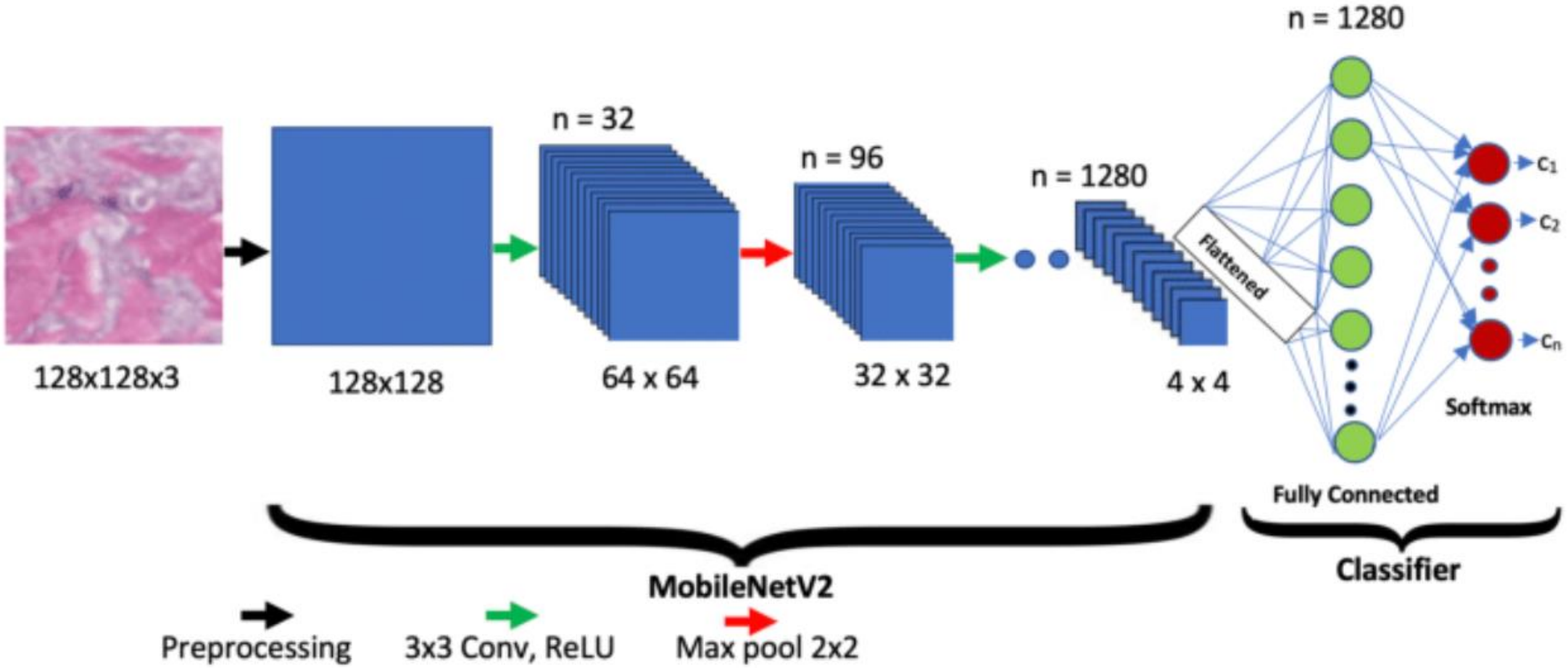
https://github.com/fchollet/deep-learning-models/blob/master/vgg19.py

LeNet-5



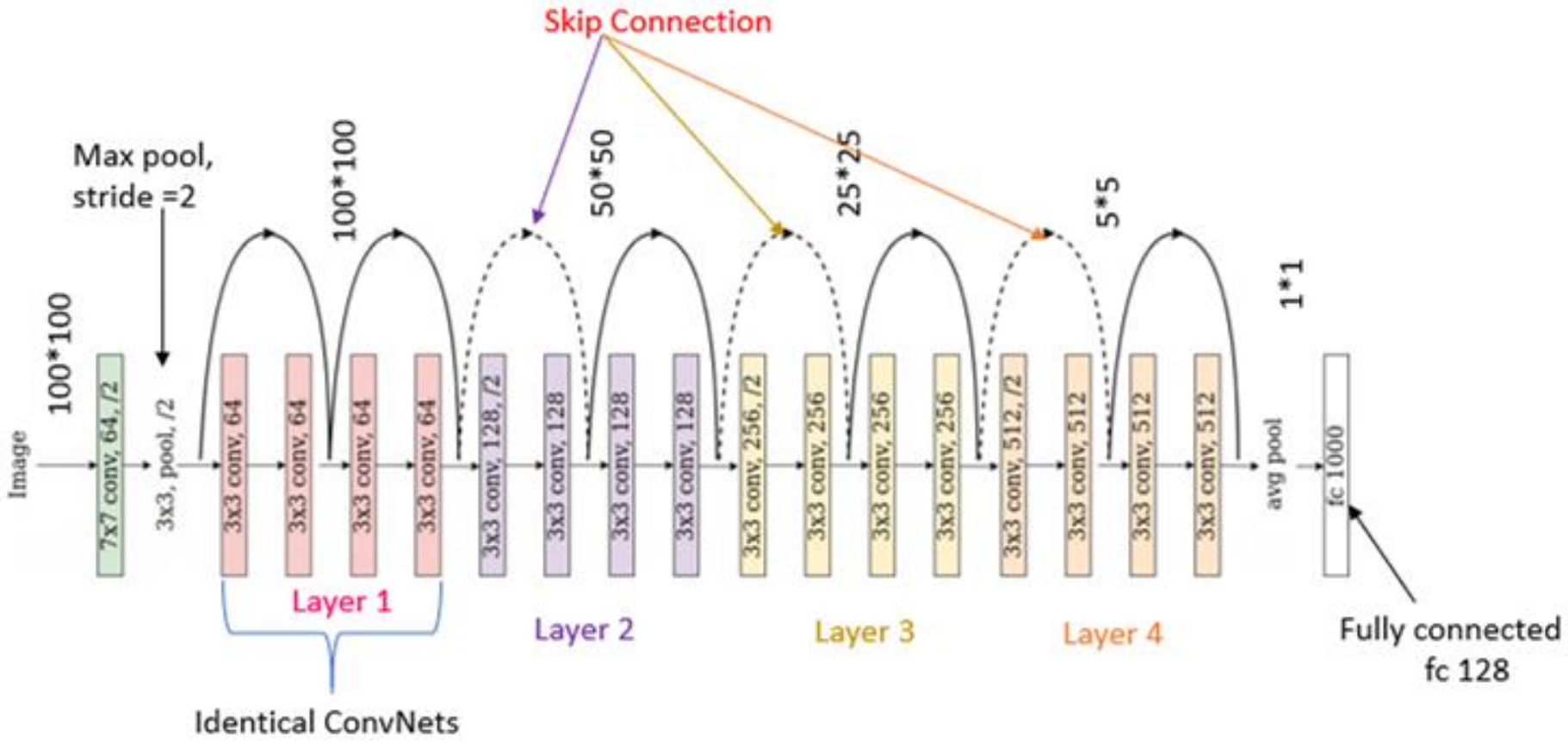
https://www.kaggle.com/code/blurredmachine/lenet-architecture-a-complete-guide/notebook

MobileNetV2



https://github.com/keras-team/keras/blob/v2.10.0/keras/applications/mobilenet_v2.py#L97-L488

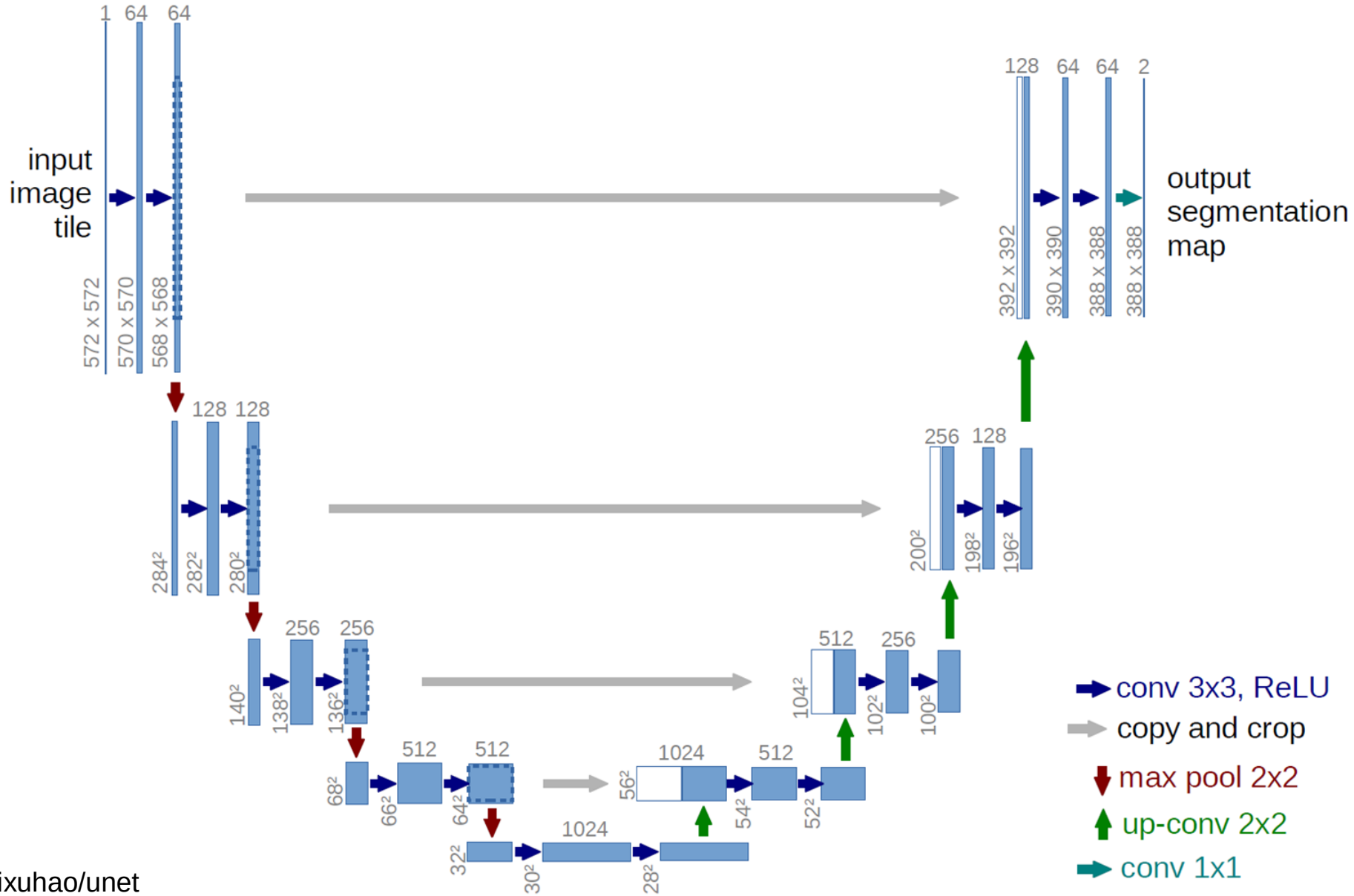
ResNet18



https://www.kaggle.com/code/dumavit1/resnet18

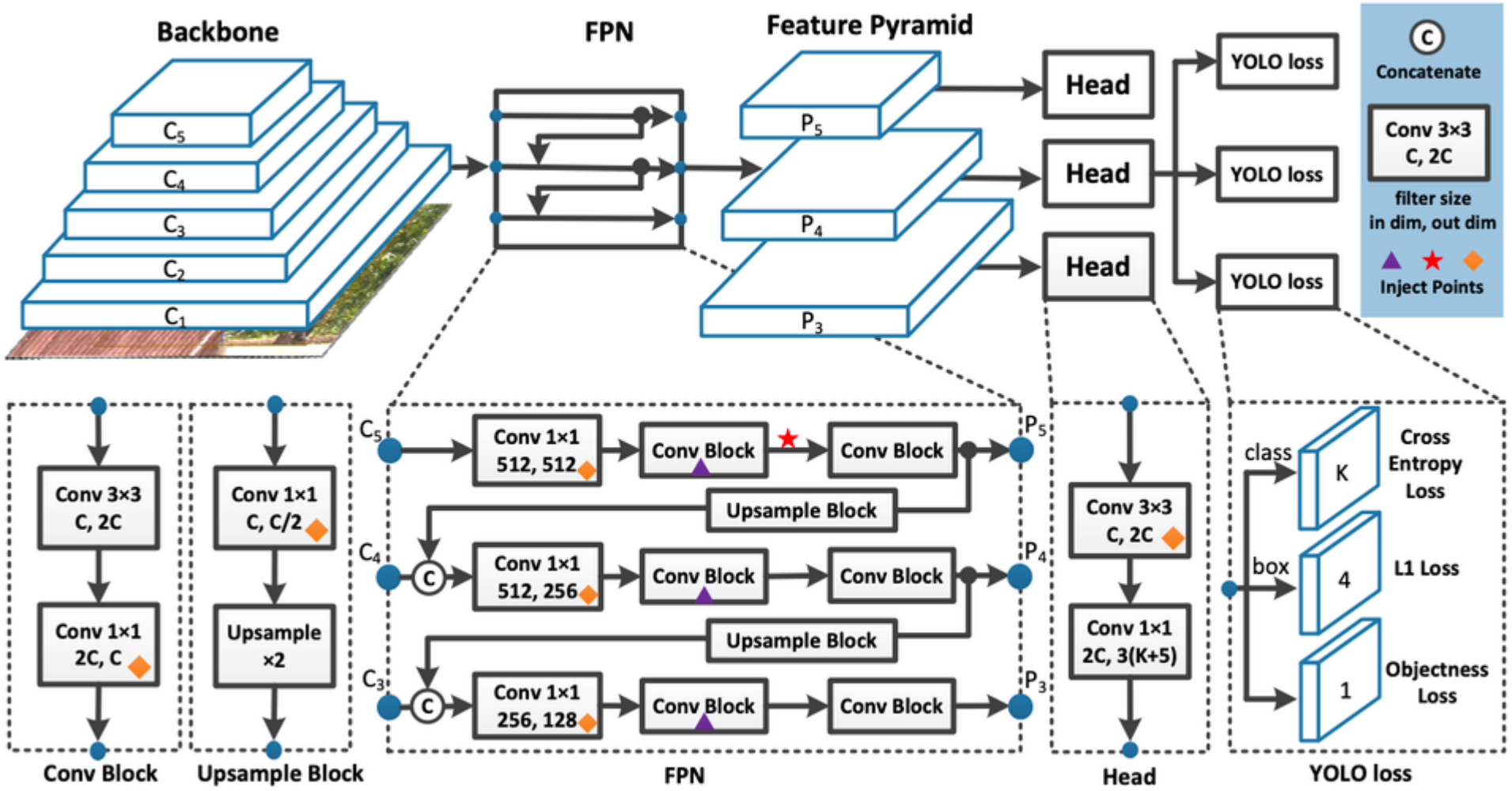
U-Net

TUSUR UNIVERSITY



https://github.com/zhixuhao/unet

YOLOv7



https://github.com/WongKinYiu/yolov7

ТУСУР
TUSUR UNIVERSITY

Метрики оценивания качества сегментации изображений и их сравнение с базовыми метриками качества решения задач с применением машинного обучения

Accuracy, precision, recall, F1-score

ТУСУР
TUSUR UNIVERSITY

Точность (Accuracy) – метрика, описывающая общую работу модели по всем классам. Напрямую зависит от сбалансированности классов.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{Correct}{All}$$

Precision – метрика, показывающая отношение числа правильно распознанных образов, к общему числу положительных образов, в не зависимости от их правильности.

$$\mathrm{Precision} = \frac{TP}{TP + FP}$$

Recall – метрика, показывающая отношение правильно классифицированных положительных образов к общему числу положительных образов. Данная метрика показывает способность модели распознавать положительные образы.

$$Recall = \frac{TP}{TP + FN}$$

F1-score представляет собой комбинацию метрик Precision и Recall, для устранения проблемы несбалансированности классов, и рассчитывается как гармоническое среднее между данными метриками.

$$\mathrm{F1} = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Pixel Accuracy

ТУСУР
TUSUR UNIVERSITY

Метрика pixel accuracy (сходимость пикселей), рассчитывается как отношение правильно распознанных пикселей к общему количеству пикселей.

$$PixAcc = \frac{Predicted\ pixels}{Ground\ truth\ pixels}$$

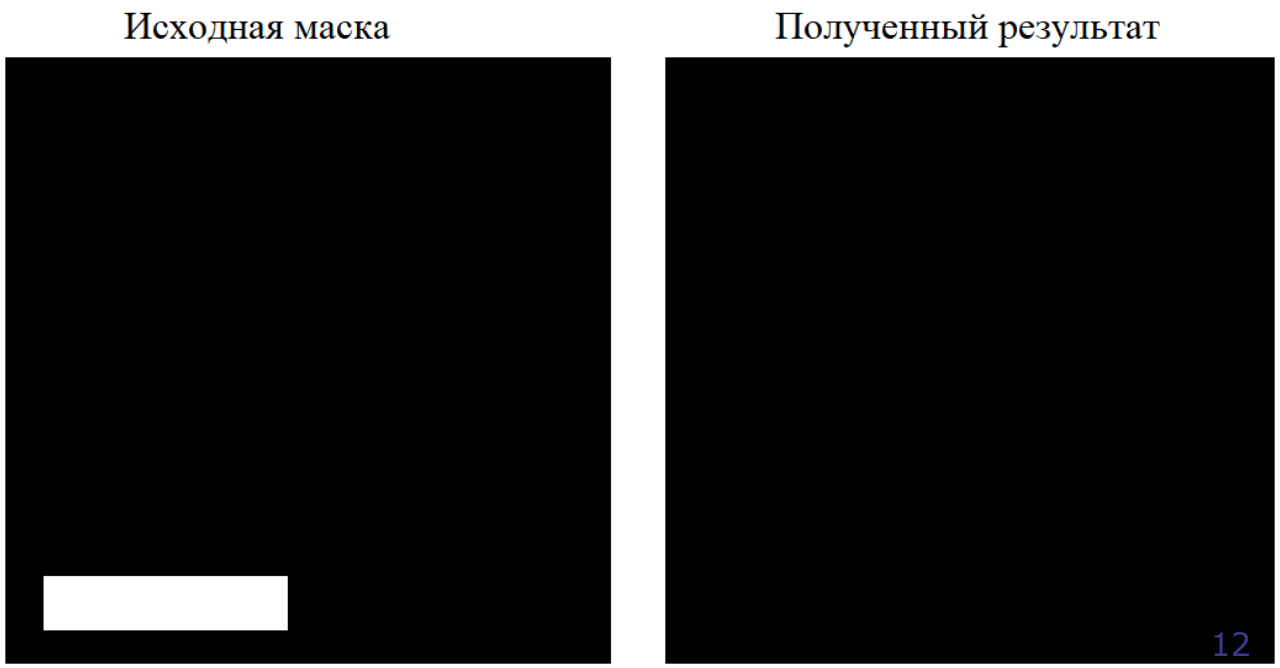
Достоинства:

- простота расчёта.

Недостатки:

- при наличии неравенства классов, метрика не может считаться достоверной

$$PixAcc = \frac{90\%}{100\%} = 90\%$$



Пересечение областей (IoU)

Пересечения областей (Intersection over Union, IoU) или индекс Джакарда (Jacard index) является одним из наиболее распространённых методов оценки точности сегментации.

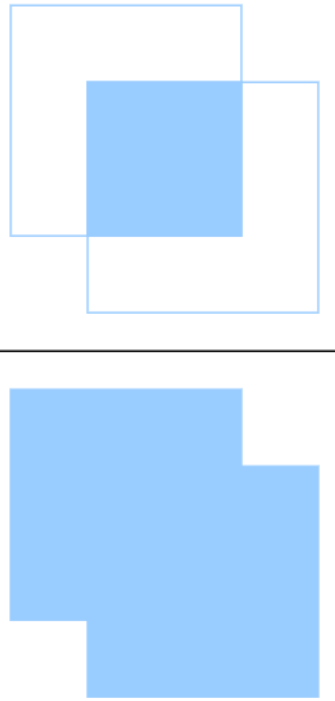
Расчёт данной метрики происходит как отношение области перекрытия оригинального объекта и распознанного объекта, к общей области объединения.

$$BgIoU = \frac{90\%}{(90+100\%)-90\%} = 90\%$$

$$SegIoU = \frac{0\%}{(10+0\%)-0\%} = 0\%$$

$$IoU = \frac{90\%+0\%}{2} = 45\%$$

$$\text{IoU} = \frac{\text{Область перекрытия}}{\text{Область объединения}}$$



Dice Coefficient

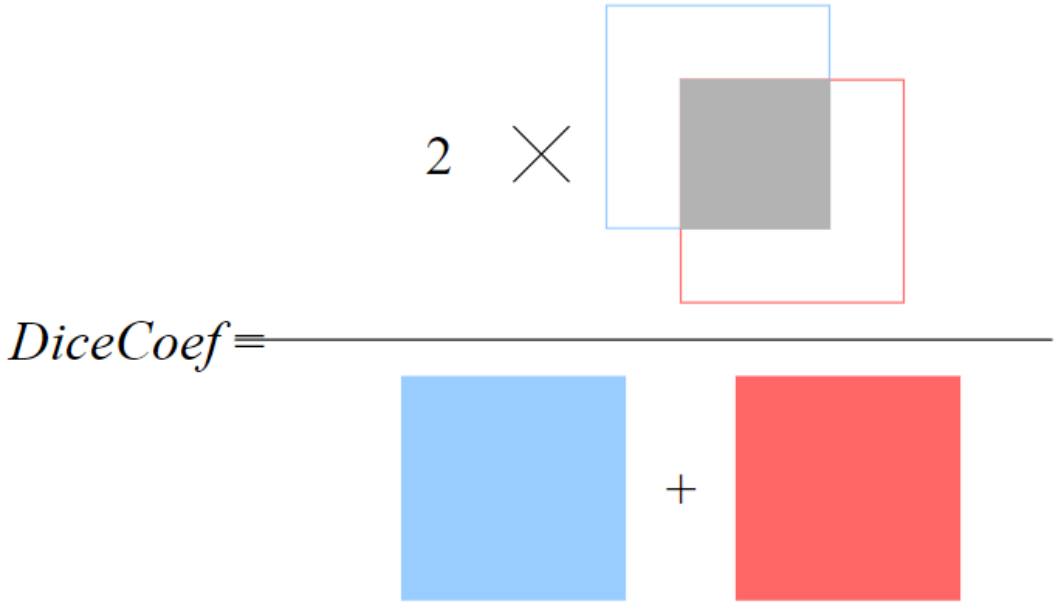
Метрика Dice coefficient рассчитывается как отношение двойной области перекрытия оригинального объекта и распознанного объекта, к сумме пикселей всех входящих во все классы.

Данную метрику также можно встретить под названием коэффициента Сёренса.

$$BgDiceCoef = \frac{2 \cdot 90\%}{200\%} = 90\%$$

$$SegDiceCoef = \frac{2 \cdot 0\%}{2000\%} = 0\%$$

$$DiceCoef = \frac{90\% + 0\%}{2} = 45\%$$



Average Precision(AP, mAP)

ТУСУР
TUSUR UNIVERSITY

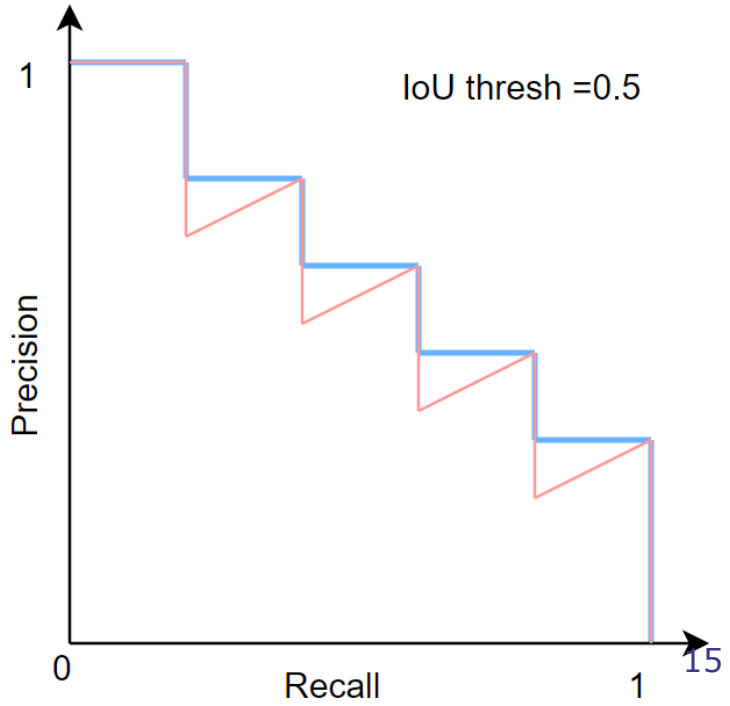
Average Precision (AP) – можно представить, как площадь под PR (Precision-Recall) кривой, полученной за счёт введения среза по значению IoU. Однако, поскольку кривая PR имеет зубчатую форму, необходимо, привести её к ступенчатой, заменив каждый подъём Precision на максимальное значение в данном «зубе».

Срез по IoU может принимать как фиксированные значения (0,5; ,0,75; 0,9), так и находиться в определённом диапазоне – [0,5; 0,95] (метрика COCO, Common Objects in Context).

Mean Average Precision (mAP) вычисляется как среднее арифметическое AP для каждого из *n* классов.

$$AP = \int_0^1 P(R)dR = \frac{1}{11}\sum_{R=0,0}^{1,0} P(R)$$

$$mAP = \frac{1}{n}\sum_{i=1}^{n} AP_i$$



ТУСУР
TUSUR UNIVERSITY

Основные алгоритмы обучения нейронных сетей при решении задач сегментации

Обучение с учителем (контролируемое обучение)

ТУСУР
TUSUR UNIVERSITY

Процесс обучения с учителем представляет собой предъявление сети выборки обучающих примеров. Каждый образец подается на входы сети, затем проходит обработку внутри структуры НС, вычисляется выходной сигнал сети, который сравнивается с соответствующим значением целевого вектора, представляющего собой требуемый выход сети. Затем по определенному правилу вычисляется ошибка, и происходит изменение весовых коэффициентов связей внутри сети в зависимости от выбранного алгоритма. Векторы обучающего множества предъявляются последовательно, вычисляются ошибки и веса подстраиваются для каждого вектора до тех пор, пока ошибка по всему обучающему массиву не достигнет приемлемо низкого уровня.

Типовые входные данные:

- Описывающие признаки;
- Матрица расстояний;
- Временной ряд;
- Изображения;
- И др.

Типовые выходные данные

- В задачах регрессии – векторы и числа бесконечной размерности;
- В задачах классификации и распознании образов – конечное число классов;
- В задачах прогнозирования – возможные результаты процесса.

Обучение с учителем

Модели контролируемого обучения можно использовать для задач:

- Распознавание изображений и объектов
- Прогнозная аналитика
- Анализ настроений клиентов
- Обнаружение спама

Недостатки:

- Модели контролируемого обучения могут потребовать определенного уровня знаний для точной структуры.
- Обучение моделей может занять очень много времени.
- Наборы данных могут иметь более высокую вероятность человеческой ошибки, что приводит к неправильному обучению алгоритмов.
- В отличие от моделей обучения без учителя, обучение с учителем не может кластеризовать или классифицировать данные самостоятельно.

Алгоритмы

ТУСУР
TUSUR UNIVERSITY

Для обучения с учителем, используются следующие алгоритмы:

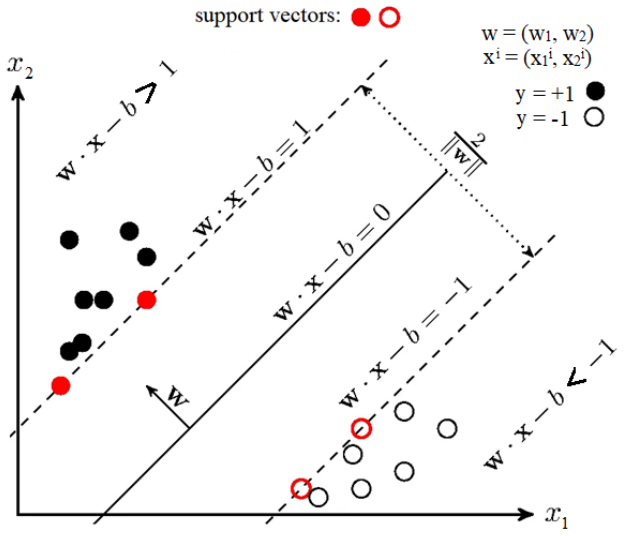
- Метод опорных векторов
- Линейная регрессия
- Логистическая регрессия
- Наивный байесовский классификатор
- Линейный дискриминантный анализ
- Дерево решений
- Алгоритм *k*-ближайших соседей
- Обучение схожести

Метод опорных векторов (SVM)

ТУСУР
TUSUR UNIVERSITY

SVM – модель, выполняющая задачи классификации и регрессии.

Суть данного метода состоит в разделении гиперплоскости в пространстве оптимальным способом так, чтобы все объекты определённого типа, были отделены от других объектов. При этом сами объекты разделяются так, чтобы они лежали в плоскости как можно дальше от границ, а объекты находящиеся ближе всех к границе и будут являться опорными векторами.



Существует два типа алгоритм SVM:

- SVM с жёстким зазором (hard-margin SVM) – когда объектам не разрешается находиться на границе разделения, т.е. не допускаются ошибки определения.
- SVM с жёстким зазором (soft-margin SVM) – когда объекты могут попадать на границу разделения, т.е. допускаются минимальные ошибки определения.

Сегментация с помощью данного метода также сводится к задаче классификации. Однако, в качестве рассматриваемых объектов представляют пиксели изображения, каждый из которых с помощью SVM относится к тому или иному классу.

Пример SVM

TUSUR UNIVERSITY

```
edge_roberts = roberts(img)
edge_roberts1 = edge_roberts.reshape(-1)
df['Roberts'] = edge_roberts1
edge_sobel = sobel(img)
edge_sobel1 = edge_sobel.reshape(-1)
df['Sobel'] = edge_sobel1
edge_scharr = scharr(img)
edge_scharr1 = edge_scharr.reshape(-1)
df['Scharr'] = edge_scharr1
edge_prewitt = prewitt(img)
edge_prewitt1 = edge_prewitt.reshape(-1)
df['Prewitt'] = edge_prewitt1
from scipy import ndimage as nd
```

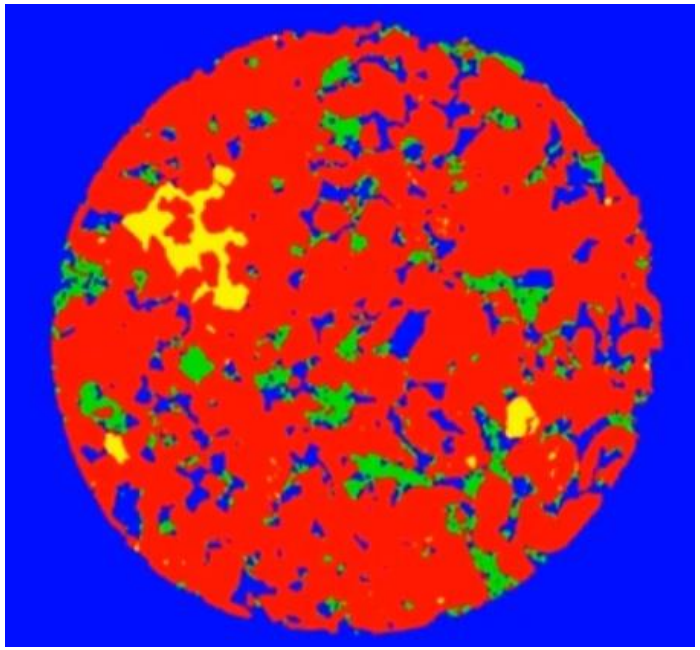
```
gaussian_img = nd.gaussian_filter(img, sigma=3)
gaussian_img1 = gaussian_img.reshape(-1)
df['Gaussian s3'] = gaussian_img1
#GAUSSIAN with sigma=7
gaussian_img2 = nd.gaussian_filter(img, sigma=7)
gaussian_img3 = gaussian_img2.reshape(-1)
df['Gaussian s7'] = gaussian_img3
#MEDIAN with sigma=3
median_img = nd.median_filter(img, size=3)
median_img1 = median_img.reshape(-1)
df['Median s3'] = median_img1
#VARIANCE with size=3
variance_img = nd.generic_filter(img, np.var, size=3)
variance_img1 = variance_img.reshape(-1)
df['Variance s3'] = variance_img1
```

Пример работы

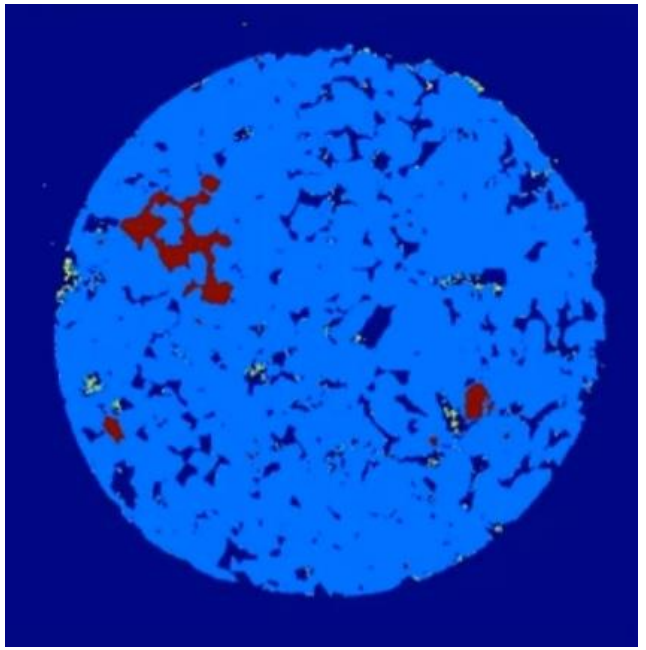
ТУСУР
TUSUR UNIVERSITY

```
Y = df["Labels"].values
X = df.drop(labels = ["Labels"], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.4, random_state=20)
from sklearn.svm import LinearSVC
model = LinearSVC(max_iter=100)
model.fit(X_train, y_train)
```

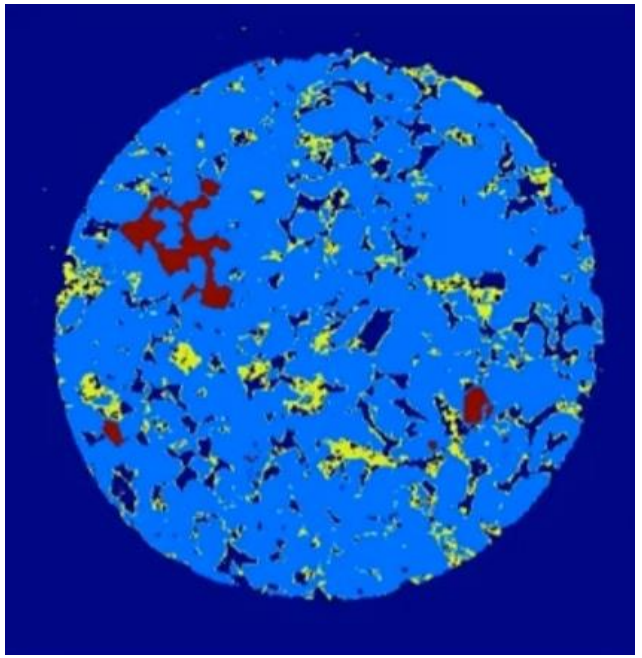
Оригинал



Обучение на 100 эпохах



Обучение на 1000 эпохах



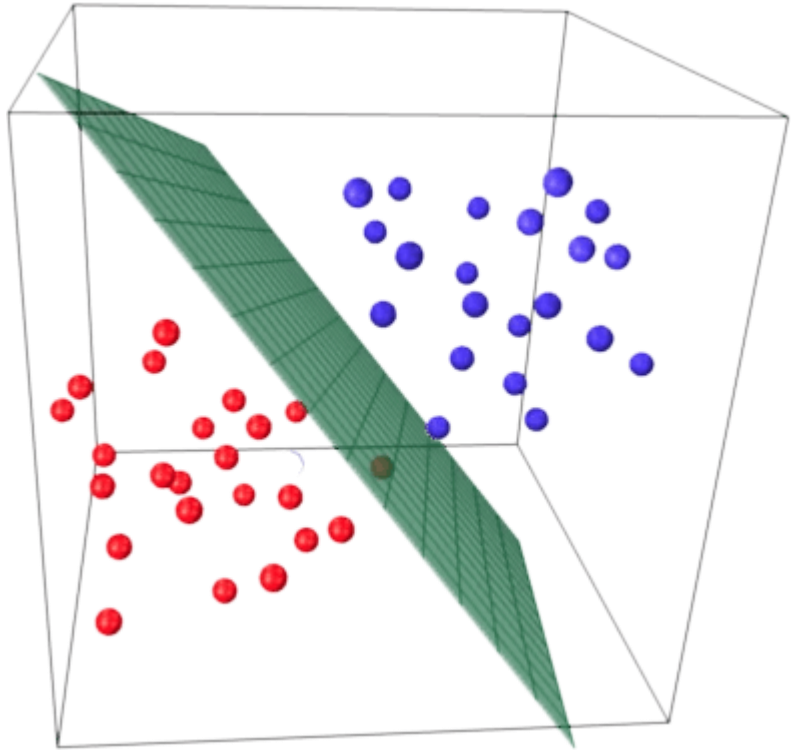
https://github.com/bnsreenu/python_for_microscopists/blob/master/068b-ML_06_04_TRAIN_ML_segmentation_All_filters_RForest_SVM.py

Логистическая регрессия

ТУСУР
TUSUR UNIVERSITY

Метод логистической регрессии по своей сути похож на метод опорных векторов. Однако, вместо простого разделения объектов на классы с помощью границы, данный алгоритм рассчитывает вероятность того, что объекты принадлежат к одному из рассматриваемых классов.

Данный метод обычно используется для задачи классификации объектов на основе их независимых параметров. При этом, хоть лучше всего он показывает себя при двух параметрах, но может использоваться и при большем количестве параметров. Так на изображении показано разделение условных объектов, имеющих три параметра.



Пример логистической регрессии

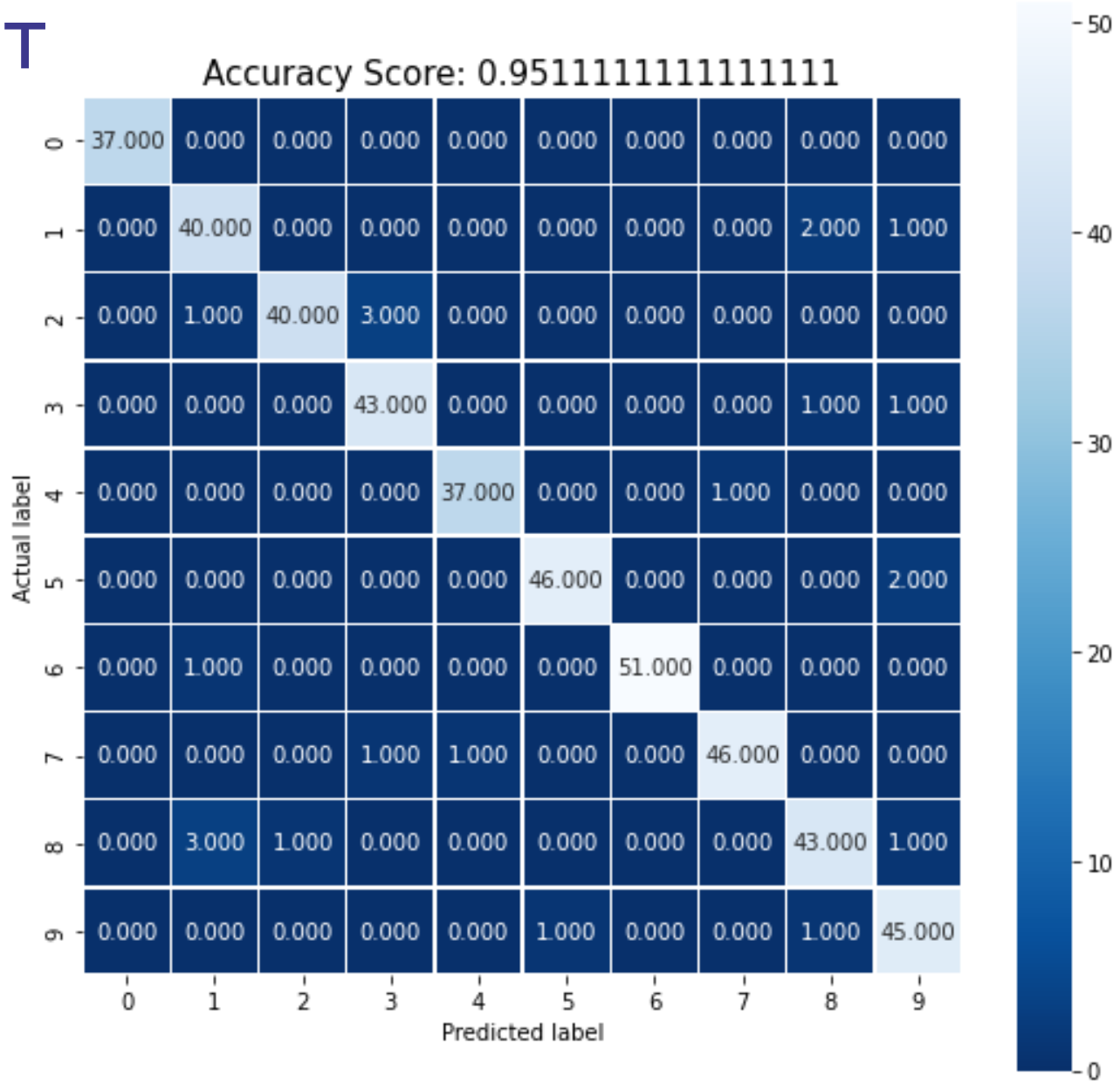
```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
digits = load_digits()
x_train, x_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.25,
 random_state=0)
```

Пример логистической регрессии

ТУСУР
TUSUR UNIVERSITY

```
logisticRegr = LogisticRegression()
logisticRegr.fit(x_train, y_train)
logisticRegr.predict(x_test[0].reshape(1,-1))
logisticRegr.predict(x_test[0:10])
predictions = logisticRegr.predict(x_test)
score = logisticRegr.score(x_test, y_test)
cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9,9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=.5, square = True, cmap = 'Blues_r');
plt.ylabel('Actual label');
plt.xlabel('Predicted label');
all_sample_title = 'Accuracy Score: {0}'.format(score)
plt.title(all_sample_title, size = 15);
```

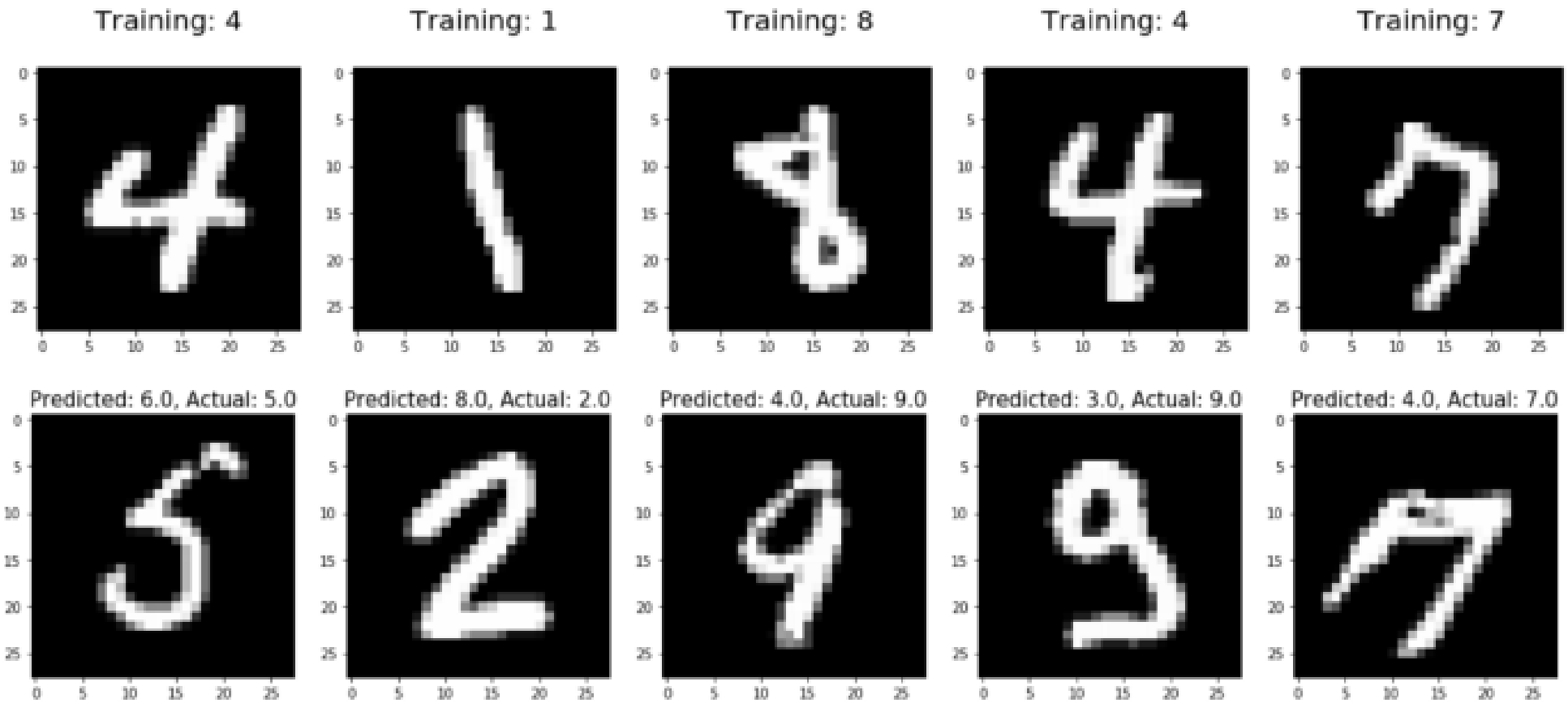
Результат



https://towardsdatascience.com/logistic-regression-using-python-sklearn-numpy-mnist-handwriting-recognition-matplotlib-a6b31e2b166a

Результат

ТУСУР
TUSUR UNIVERSITY



Наивный байесовский классификатор

Данный метод строится на подходе, основанном на независимости признаков объектов согласно теореме Байеса, т.е. никакой из признаков не может повлиять на появление или отсутствие другого.

Именно то, что признаки являются независимыми друг от друга, и делает такой подход «наивным».

Существует несколько разновидностей данного метода:

- Многокомпонентный наивный байесовский классификатор (Multinomial Naïve Bayes);
- Наивный байесовский классификатор Бернулли (Bernoulli Naïve Bayes);
- Наивный байесовский классификатор Гаусса (Gaussian Naïve Bayes).

Результатом этого метода является вероятность принадлежности объекта к рассматриваемым классам.

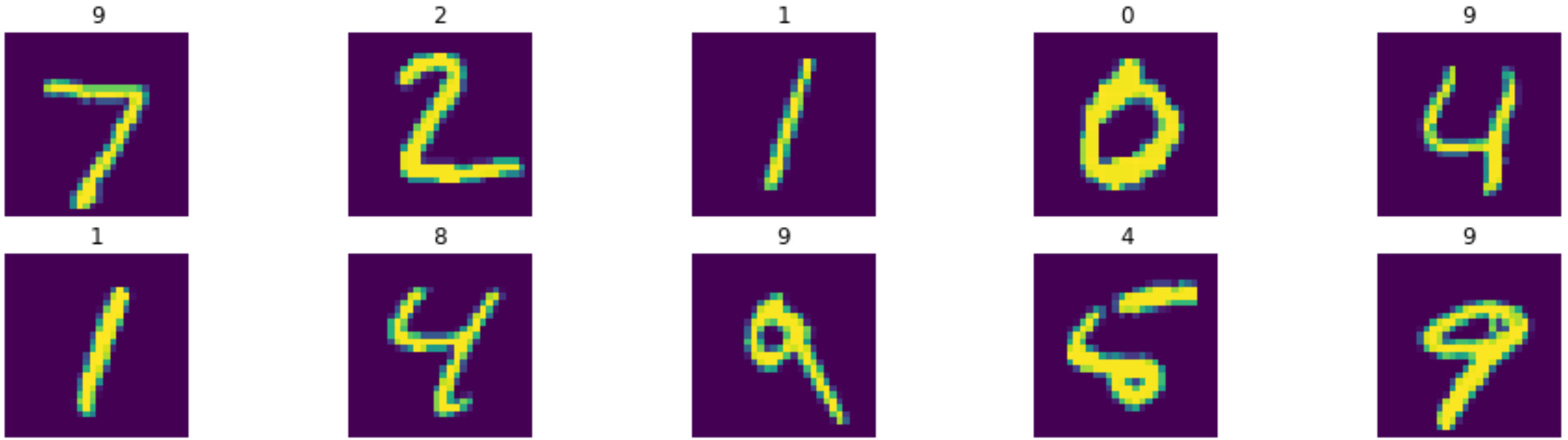
Поскольку данный метод будет рассматривать признаки объектов (в нашем случае пикселей изображения) как не зависимые параметры, результаты распознавания объектов могут быть низкими. Связанно это с тем, что в реальности наличие определённых схожестей и пересечений признаков у объектов является естественным, и они в действительности могут порождать друг друга, что не учитывается в данном методе.

Пример наивного байесовского классификатора

ТУСУР
TUSUR UNIVERSITY

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
import seaborn as sns
gnb = GaussianNB()
(train_X, train_y), (test_X, test_y) = mnist.load_data()
X, y = train_X.reshape(60000,784), train_y
y_pred = gnb.fit(X, y).predict(test_X.reshape(10000,784))
cm = metrics.confusion_matrix(test_y, y_pred)
pic_box = plt.figure(figsize=(16,4))
for i in range(10):
  pic_box.add_subplot(2,5,i+1)
  plt.title(y_pred[i])
  plt.imshow(test_X[i])
  plt.axis('off')
plt.show()
```

Результат работы



https://scikit-learn.org/stable/modules/naive_bayes.html

Древо решений

ТУСУР
TUSUR UNIVERSITY

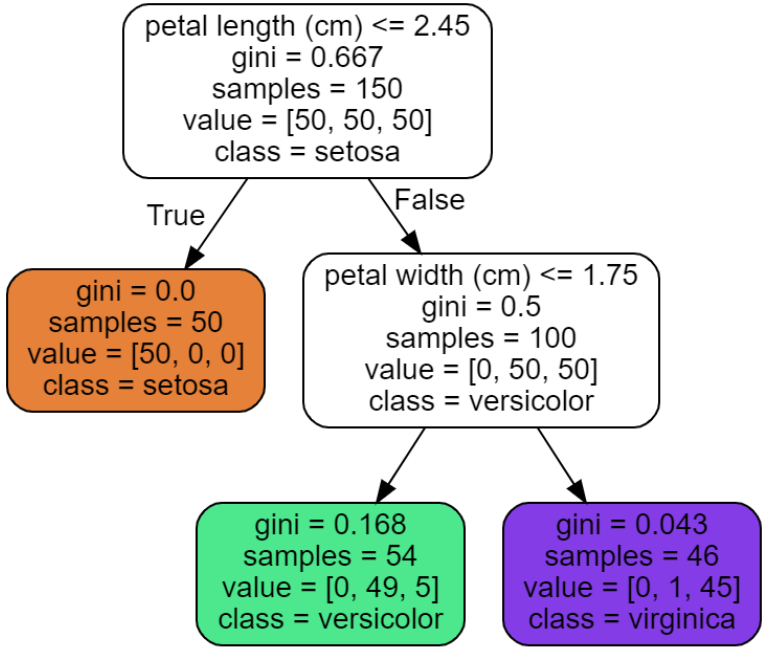
Метод дерева решений базируется на проверке объекта по признакам, используя простые условия типа «Если *условие*, то *результат*».

В данном дереве имеется три вида блоков:

- Корень – определяет начальное условие;
- Дочерний узел – определяет дополнительные условия;
- Лепесток – обозначает один из определённых классов.

Главной особенностью и достоинством данного метода является то, что им не требуется какая-либо предобработка данных, что может положительно сказаться в задаче сегментирования и классификации объектов на изображении, где классы могут быть плохо сбалансированы.

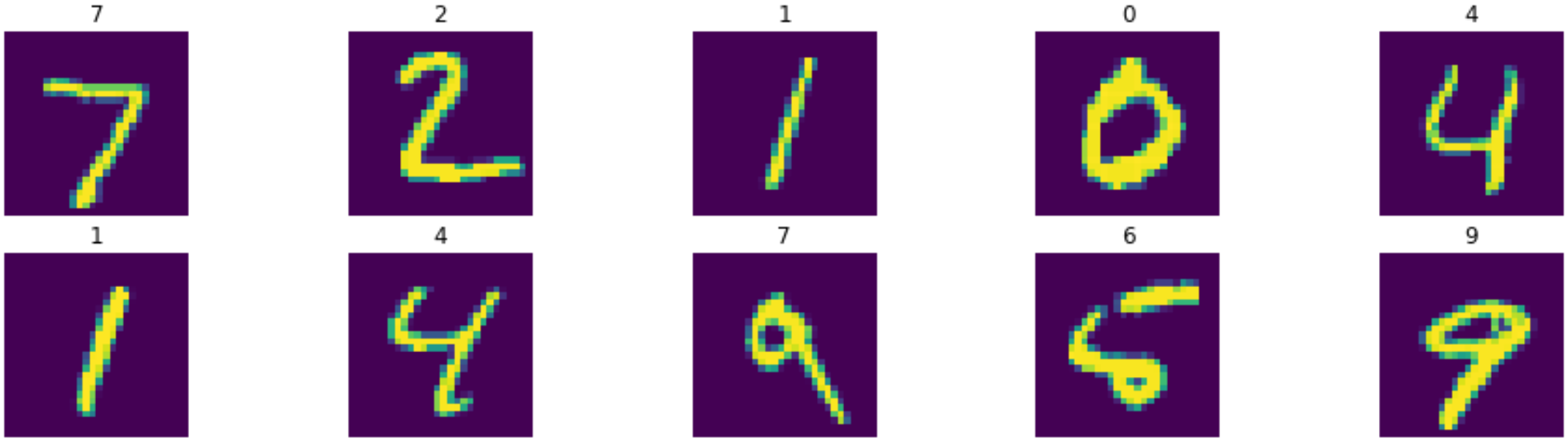
Самой простой реализацией в сегментации изображении может является проверка цветового оттенка пикселя: зелёный – трава, синий – небо и т.д.



Пример

```
import matplotlib.pyplot as plt
#Loading
from keras.datasets import mnist
(train_X, train_y), (test_X, test_y) = mnist.load_data()
X, y = train_X.reshape(60000,784), train_y
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, y)
pred = clf.predict(test_X.reshape(10000,784))
pic_box = plt.figure(figsize=(16,4))
for i in range(10):
  pic_box.add_subplot(2,5,i+1)
  plt.title(pred[i])
  plt.imshow(test_X[i])
  plt.axis('off')
plt.show()
```

Результат



https://biconsult.ru/products/derevya-resheniy-v-mashinnom-obuchenii

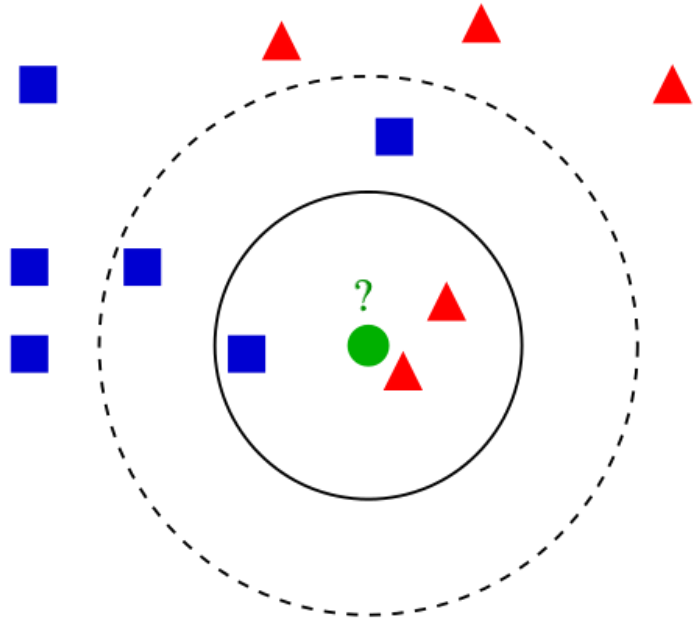
Метод k-ближайших соседей

ТУСУР
TUSUR UNIVERSITY

Суть данного метода заключается в определении класса объекта на основе уже известных классов его ближайших соседей. При этом в случае классификации присваивается класс, который будет преобладать среди соседей, а в задаче регрессии будет использоваться среднее значение среди соседей.

Алгоритм сегментации изображения:

1. Выбираем количество кластеров (классов объектов), которые необходимо выделить;
2. Случайным образом выбираем значение центройдов для кластеров;
3. Задаём количество рассматриваем ближайших соседей;
4. Классифицируем значения каждого пикселя к ближайшему центройду, для определения первичных кластеров;
5. Пересчитываем значения центройдов для каждого кластера;
6. Проверяем значения пикселей в каждом кластере, относительно новых значений центройдов и в случае необходимости корректируем классификацию.

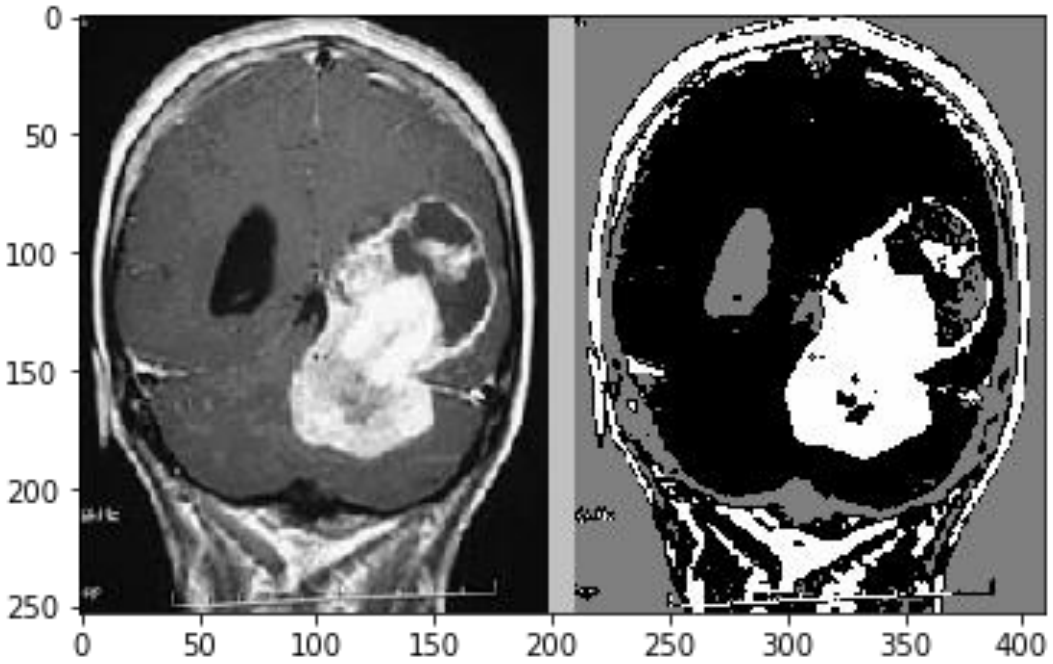


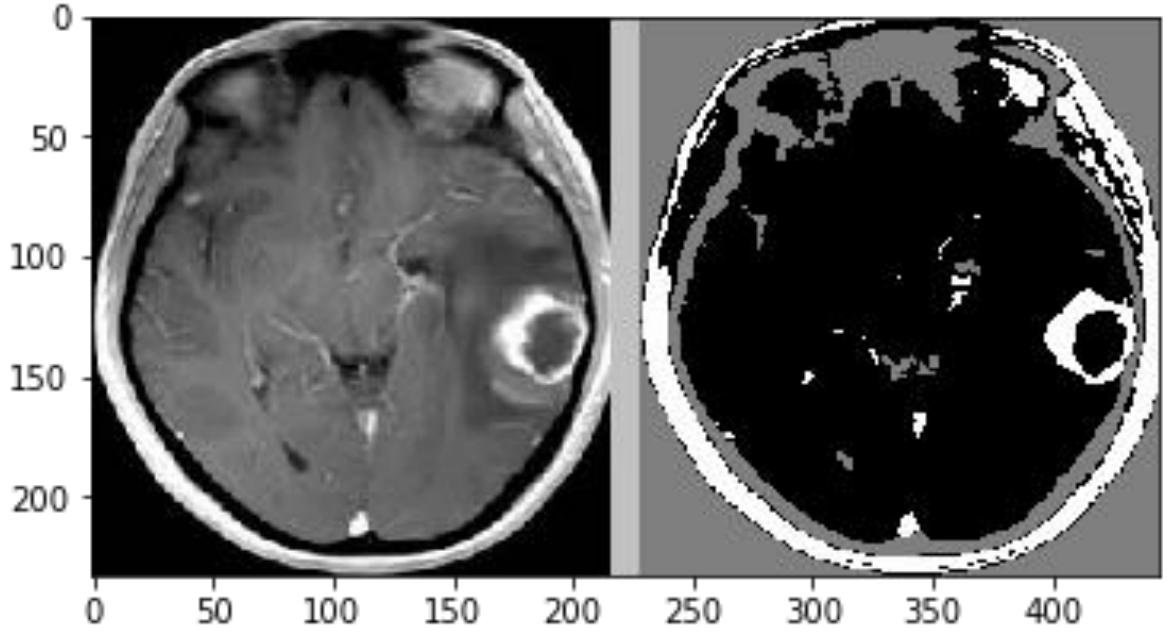
Пример

ТУСУР
TUSUR UNIVERSITY

```
import numpy as np
from sklearn.cluster import KMeans
import cv2
import datetime
from matplotlib import pyplot as plt
import time
for i in range(0,3):
    reshaped[i] = image[i].reshape(image[i].shape[0] * image[i].shape[1], image[i].shape[2])
clustering=[0,0,0]
for i in range(0,3):
    kmeans = KMeans(n_clusters=numClusters[i], n_init=40, max_iter=500).fit(reshaped[i])
    clustering[i] = np.reshape(np.array(kmeans.labels_, dtype=np.uint8),
    (image[i].shape[0], image[i].shape[1]))
sortedLabels=[[],[],[]]
for i in range(0,3):
    sortedLabels[i] = sorted([n for n in range(numClusters[i])],
        key=lambda x: -np.sum(clustering[i] == x))
kmeansImage=[0,0,0]
concatImage=[[],[],[]]
for j in range(0,3):
    kmeansImage[j] = np.zeros(image[j].shape[:2], dtype=np.uint8)
    for i, label in enumerate(sortedLabels[j]):
        kmeansImage[j][ clustering[j] == label ] = int((255) / (numClusters[j] - 1)) * i
    concatImage[j] = np.concatenate((image[j],193 * np.ones((image[j].shape[0], int(0.0625 * image[j].shape[1]), 3), dtype
=np.uint8),cv2.cvtColor(kmeansImage[j], cv2.COLOR_GRAY2BGR)), axis=1)
```

Результат





https://github.com/AbhinavUtkarsh/Image-Segmentation/blob/master/KNN%20Image%20Segmentation.ipynb

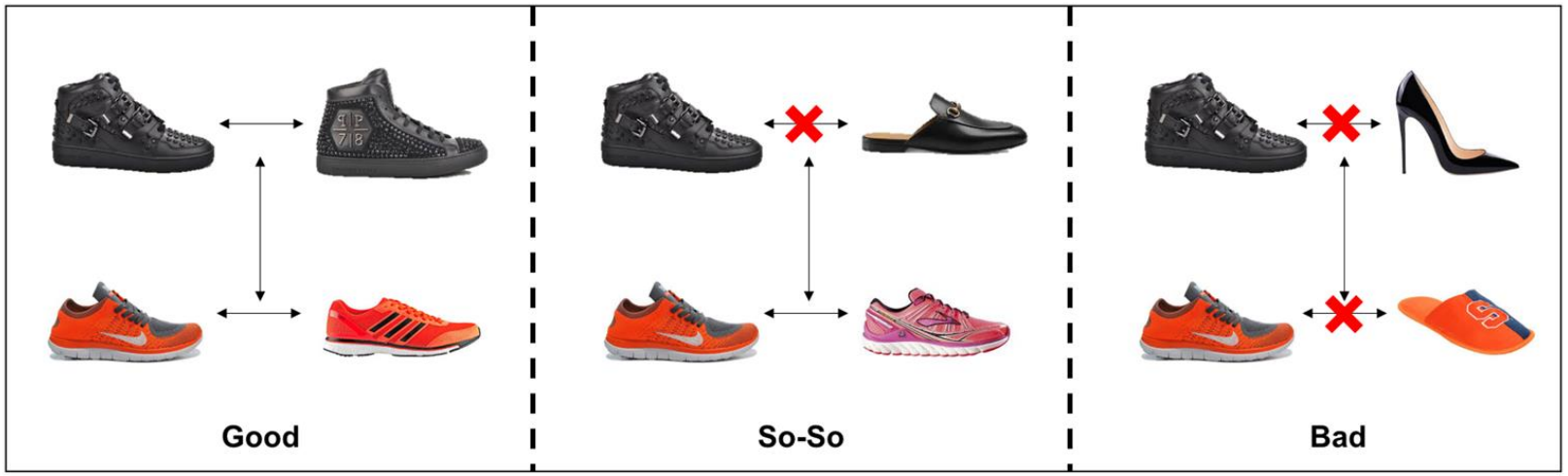
Обучение схожести

Обучение схожести базируется на построении функции подобия, которая измеряет насколько похожи между собой два объекта. В качестве параметров схожести могут рассматриваться формы объекта, цвет, положение в пространстве и др.

Общий алгоритм метода:

- Преобразование признаков в вектор;
- Сравнение векторов с помощью метрики расстояния;
- Классификация полученного расстояния, для определения схожести.

В качестве метрики расстояния можно использовать расстояние Евклида или расстояние Манхэттена.



Пример

ТУСУР
TUSUR UNIVERSITY

```
from transformers import ViTModel
from transformers.modeling_outputs import SequenceClassifierOutput
import torch.nn as nn
class ViTForImageClassification(nn.Module):
    def __init__(self, num_labels=10, vector_length=1000):
        super(ViTForImageClassification, self).__init__()
        self.vit = ViTModel.from_pretrained('google/vit-base-patch16-224-in21k')
        self.dropout = nn.Dropout(0.1)
        self.last_layer = nn.Linear(self.vit.config.hidden_size, num_labels)
        self.num_labels = num_labels
    def forward(self, pixel_values, labels):
        outputs = self.vit(pixel_values=pixel_values)
        output = self.dropout(outputs.last_hidden_state[:,0])
        logits = self.last_layer(output)
        loss = None
        if labels is not None:
          loss_fct = nn.CrossEntropyLoss()
          loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))
        return SequenceClassifierOutput(
            loss=loss,
            logits=logits,
            hidden_states=outputs.hidden_states,
            attentions=outputs.attentions,
        )
```

Пример

```
from datasets import load_metric
import numpy as np
metric = load_metric("accuracy")
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = np.argmax(predictions, axis=1)
    return metric.compute(predictions=predictions, references=labels)
trainer = Trainer(
    model,
    args,
    train_dataset=preprocessed_train_ds,
    eval_dataset=preprocessed_val_ds,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.train()
```

Результаты

Selected item



Similar items



https://colab.research.google.com/drive/17ftAEdu7diEavKGl5DbYFPFvuGKWp_2X#scrollTo=lzYq7_d5HKnw

https://towardsdatascience.com/similarity-learning-for-clothing-building-a-web-service-from-scratch-350216830e21

Обучение без учителя

ТУСУР
TUSUR UNIVERSITY

При обучении без учителя обучающее множество состоит лишь из входных векторов. Обучающий алгоритм подстраивает веса сети так, чтобы получались согласованные выходные векторы, т.е. чтобы предъявление достаточно близких входных векторов давало одинаковые выходы.

Процесс обучения, следовательно, выделяет статистические свойства обучающего множества и группирует сходные векторы в классы. Предъявление на вход вектора из данного класса даст определенный выходной вектор, но до обучения невозможно предсказать, какой выход будет производиться данным классом входных векторов.

Следовательно, выходы подобной сети должны трансформироваться в некоторую понятную форму, обусловленную процессом обучения. Это не является серьезной проблемой. Обычно не сложно идентифицировать связь между входом и выходом, установленную сетью.
Для обучения нейронных сетей без учителя применяются сигнальные метод обучения Хебба и Ойа.

Недостатки:

- Вычислительная сложность из-за большого объема обучающих данных;
- Более длительное время обучения;
- Более высокий риск неточных результатов;
- Вмешательство человека для проверки выходных переменных;
- Отсутствие прозрачности в основе, на которой были сгруппированы данные.

Алгоритмы

ТУСУР
TUSUR UNIVERSITY

Некоторые из наиболее распространенных алгоритмов, используемых в обучении без учителя, включают:

- Кластеризацию
  - Иерархическая кластеризация
  - Метод k-средних
  - Смешанные модели
  - DBSCAN
  - OPTICS
- Обнаружение аномалий
  - Фактор локального выброса
  - Изолированный лес
- Подходы к обучению моделей со скрытыми переменными.
  - Метод моментов
  - Слепое разделение сигналов

Метод k-средних

Метод k-средних – это метод кластерного анализа, целью которого является разделение m наблюдений на k кластеров, при этом каждое наблюдение относится к тому кластеру, к центру (центроиду) которого оно ближе всего.

Использование данного метода для сегментации заключается в разделении изображения на кластеры (классы объектов), согласно их цветовому оттенку.

Алгоритм сегментации:

1. Выделяем каждый слой согласно его цветовому содержанию (RGB = 3 слоя: красный, зелёный и синий);
2. Задаём количество кластеров (классов объекта), которые необходимо выделить;
3. Случайным образом выбираем значения центройдов для каждого из кластеров;
4. Сравниваем значения для каждого пикселя по всем трём слоям, и относим к одному из кластеров, центройд которого более «близкий» к значениям пикселя;
5. Корректируем значение центройда относительно среднего значения по всем пикселям в класетре;
6. Корректируем классификацию пикселей по кластерам относительно новых значений центройдов.

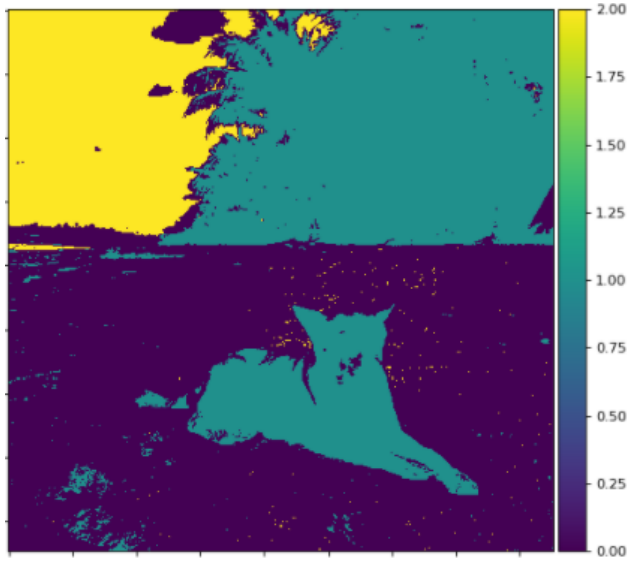
Пример с использованием k-средних

ТУСУР
TUSUR UNIVERSITY

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import colors
from skimage.color import rgb2gray, rgb2hsv, hsv2rgb
from skimage.io import imread, imshow
from sklearn.cluster import KMeans
dog = imread('beach_doggo.PNG')
plt.figure(num=None, figsize=(8, 6), dpi=80)
plt.figure(num=None, figsize=(8, 6), dpi=80)
kmeans = KMeans(n_clusters=  4, random_state = 42).fit(df_doggo)
result = kmeans.labels_.reshape(dog.shape[0],dog.shape[1])
imshow(result, cmap='viridis')
```

Результат работы











https://towardsdatascience.com/image-processing-with-python-unsupervised-learning-for-image-segmentation-90ebd23d91a4

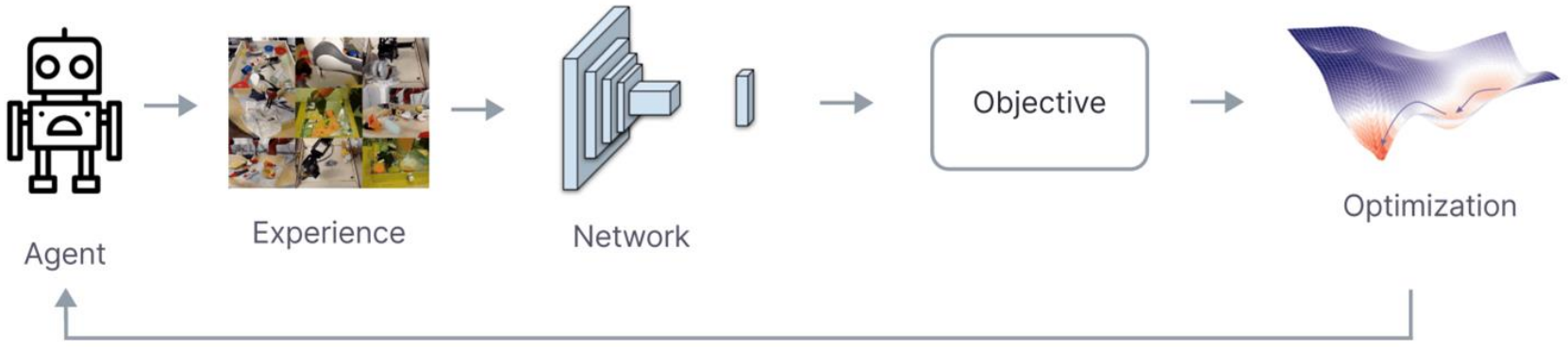
Обучение с подкреплением

Данный метод основан на получении итоговых результатов не из слепого анализа только предоставляемых данных, но и на анализе дополнительных данных получаемых из «окружающей среды». Например, такими дополнительными данными из среды могут быть время обработки, сами результаты обработки, их размер и др.

Таким образом, не только уменьшается вклад возможной ошибки исследователя, но и отсутствует большая проблема слишком долгих и требовательных процессов обучений, поскольку алгоритм сам оптимизирует подход к обучению.

Возможные решаемые задачи:

- Расширение датасета;
- Нахождение оптимальной стратегии игры;
- Распознавание объектов в реальном времени;
- Нахождение оптимального пути.

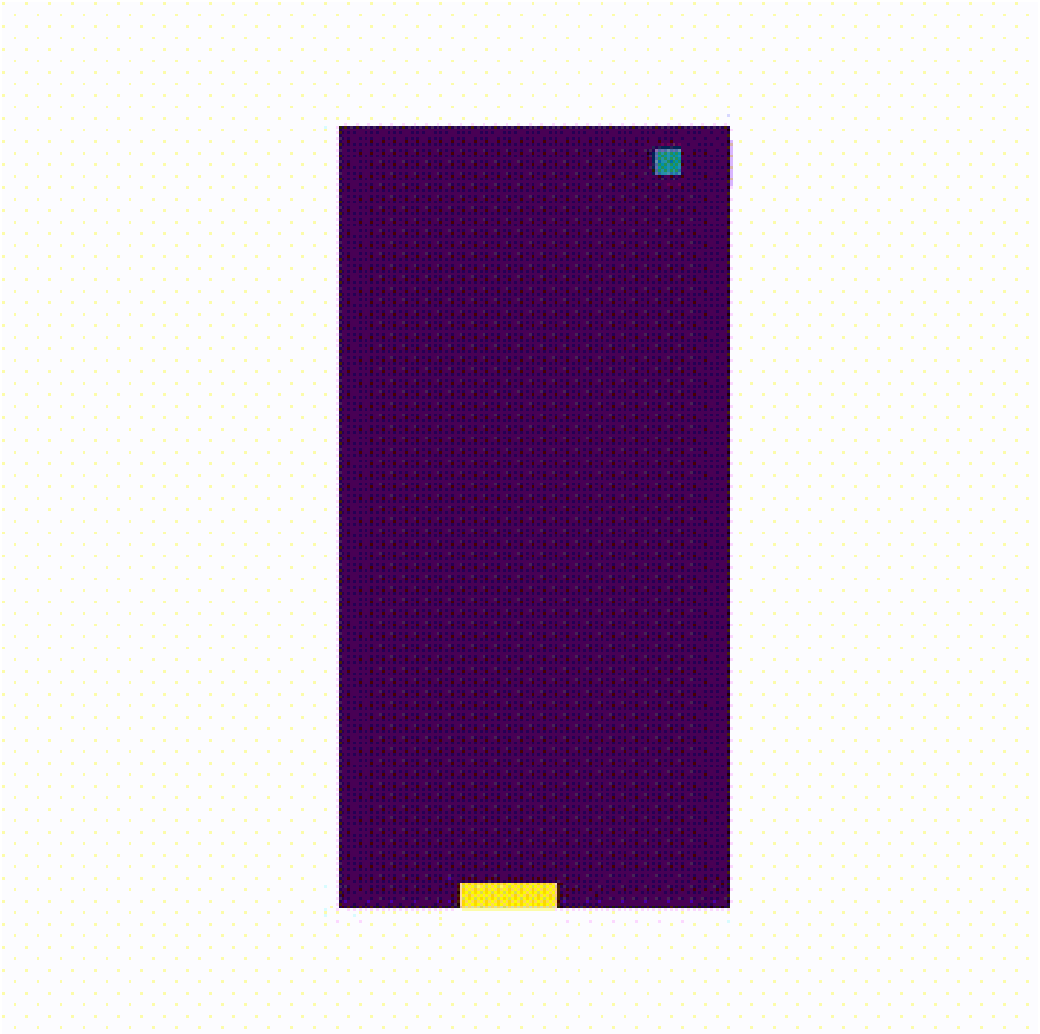


Пример обучения

ТУСУР
TUSUR UNIVERSITY

```
for x in range(0, 100000):
    print('Цикл обучения № ', x)
    data_set_images, data_set_actions, data_set_reward = game_funct(kol_game, policy_estimator = pe)
    global_buffer.append_butch(data_set_images, data_set_actions, data_set_reward)
    sel_pr = teach_net(global_buffer, policy_estimator=pe, optimizer = optimAdam, device_in=device)
    global_buffer.clear()
      if len(list_rolout_score) >= step:
        if x % 50 == 0:
            clear_output()
            print(sel_pr)
            paint_mean_score(x)
        mean_score = np.array(list_rolout_score[-step:]).mean()
        mean_score = round(mean_score, 2)
        print('Точность на последних циклах ', str(step), ' = ', mean_score)
        if mean_score >= kol_point*kol_game:
            print('Обучение Закончено')
            break
```

Пример обучения



https://habr.com/ru/post/506384/

TYCYP
TUSUR UNIVERSITY